

BUSINESS CASE	
Proposed Project	Local Desktop Password Manager
Date Produced	September 16 th , 2026
Background	In this age of online services and web applications, many websites require that users create a personal account so that they can access the features and services the application offers. Majority of applications require a user's account to be secured with a text-based password, and some method of two-factor-authentication, to ensure that only the authorized user who created the account can log into the account. This creates a strain on the user to memorize different passwords for each account and be able to recall what password was used for which account. Many users opt to either use the same password for multiple accounts or utilize dictionary to both meet general password length requirements and make the password rememberable. These both create security issues for the user, as if the user's password hash is leaked due a site's security being compromised, and is cracked, it could be used to gain access to a user's other account on another site. Furthermore, dictionary-based passwords are easy for an infiltrator to brute-force and crack.
Business Need/ Opportunity	Data and security breach happen due to password leaks. Weak, repeated and poorly managed passwords cause data leaks which leads to identity theft, data loss, financial loss and so on. There are existing solutions for the problem, but they lack security at times and some of the commercial apps includes subscription costs includes cloud subscription cost and so on. Third party cloud applications costs more and the app security is not certain. In business perspective, organizations usually adopt the local password manager applications are designed to reduce reusable passwords, reduces password resets, and supports basic security compliance within the organization. It will not let users to create weak passwords and reduce the dependency of remembering the passwords.
Options	We identified the following options: 1. Do nothing 2. Write a script that pulls usernames and passwords from a text file 3. Design a database program that stores a user's usernames and passwords in an encrypted file, leaving the user to only need to remember a single password to decrypt the database 4. Design a program that can store and produce time-based one-time

- passwords and recovery keys, enabling the user to setup 2FA on online accounts that support the security feature.
5. Combination of 3 and 4
 6. Cloud synced password manager

Cost-Benefit Analysis

Option 1: Do nothing

Benefits:

- No time or financial investment needed

Costs:

- User must rely on their own memory (forgetting passwords)
- User may sacrifice security for convenience with simple and repeated passwords
- No ability for 2FA authentication
- Higher risk of compromised account

Option 2: Script for pulling usernames and passwords from a text file

Benefits:

- Usernames and Passwords are easily accessible to the user
- Offload memorization of login details
- Search functionality

Costs:

- Time and financial investment into script development
- Passwords are stored in plain text
- Compromised device exposes passwords with ease
- No ability for 2FA authentication

Option 3: Program for storing usernames and passwords in an encrypted file/database

Benefits:

- Usernames and Passwords are accessible to the user
- User can come up with more complex and secure passwords for accounts, without worry of misremembering or forgetting passwords
- User only needs to remember one password, instead of many
- Search functionality

Costs:

- Greater time and financial investment into program design and development
- Program dependent on user to remember the password to decrypt and access the password database
- No ability for 2FA authentication

Option 4: Program that can store and produce time-based one-time passwords

Benefits:

- Ability to setup 2FA authentication for better account security
- Access to recovery keys in case user cannot authenticate with 2FA
- User can setup accounts to be password-less via 2FA (dependent on service supporting password-less accounts)
- Search functionality
- Better market-share by supporting 2FA security approaches.

Costs:

- Greater time and financial investment into program design and development
- Regular password/account information cannot be stored

Option 5: Program for storing usernames, text passwords, and time-based one-time passwords in an encrypted file/database

Benefits:

- Usernames and Passwords are accessible to the user
- User can come up with more complex and secure passwords for accounts, without worry of misremembering or forgetting passwords
- User only needs to remember one password, instead of many
- Search functionality
- Ability to setup 2FA authentication for better account security
- Access to recovery keys in case user cannot authenticate with 2FA
- User can setup accounts to be password-less via 2FA (dependent on service supporting password-less accounts)
- Better market-share by supporting 2FA security approaches.

Costs:

- Greatest time and financial investment into program design and development
- High-level of trust placed onto program, needs to be of high-quality and standards

Option 6: Cloud synced password manager

Benefits:

- User can access usernames and passwords across multiple devices
- encrypted database is stored on server, not on user's device

Costs:

- Time and financial investment into program design and development
- Additional time and financial investment in networking and secure data transfer support
- User loses access to passwords and usernames if either the user or the server is offline
- Malicious actor can compromise server, or user's account to steal login details

Recommendation

Option 5: Program for storing usernames, text passwords, and time-based one-time password keys and recovery keys in an encrypted file/database

Benefits:

- Usernames and Passwords are accessible to the user
- User can come up with more complex and secure passwords for accounts, without worry of misremembering or forgetting passwords
- User only needs to remember one password, instead of many
- Search functionality
- Ability to setup 2FA authentication for better account security
- Access to recovery keys in case user cannot authenticate with 2FA
- User can setup accounts to be password-less via 2FA (dependent on service supporting password-less accounts)
- Better market-share by supporting 2FA security approaches.

Costs:

- Greatest time and financial investment into program design and development
- High-level of trust placed onto program, needs to be of high-quality and standards

